

Agenda

01

Session 1: Spec-Driven Development

Foundations, structural alignment, and automated context refinement for spec-driven workflows.

SPEAKERS:

Priyam Gupta & Shashwat Rawat

02

Session 2: Efficient AI Engineering

Optimizing context management, agentic skills, workflows, and system observability.

SPEAKERS:

Yash Sinha & Pratikkumar Mohite

03

Session 3: Devs.ai in Action

Live demonstration of Devs.ai—a no-code agentic AI and app development platform.

SPEAKERS:

Uday Girhepunje



Phase 1: Spec Driven Development

AI Tech Meetup India 2026

Priyam Gupta & Shashwat Rawat | 10th Oct 2026

Don't prompt a marketplace. Spec it. Then let agents ship.

A live hour of Spec-Driven Development — architecture and UI already in Figma, design spec written on stage, then a running AppStore if the hour allows.

PHASE 1

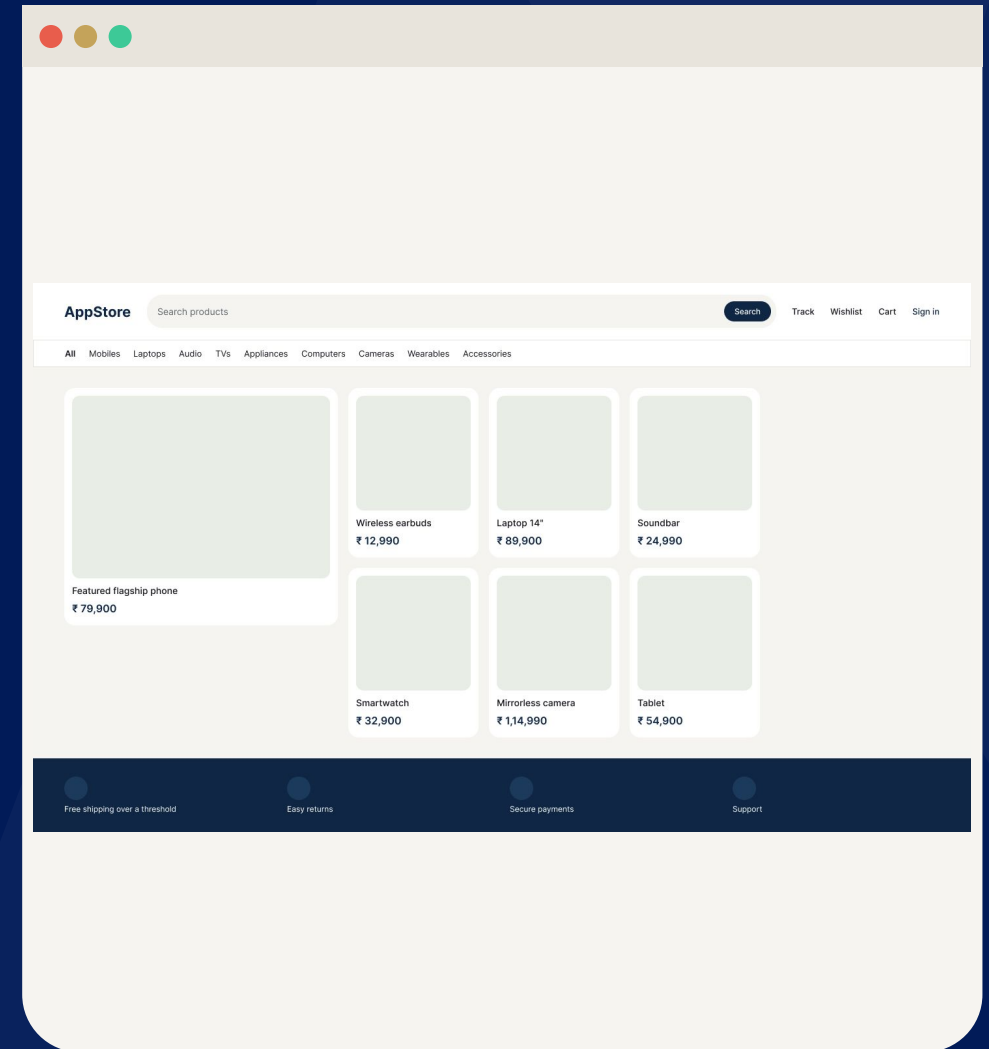
Spec-Driven
Development

LIVE BUILD

AppStore marketplace

THE PROOF

docker compose up



Four beats. No filler.

01 The inputs

Architecture board and UI screens — already designed.



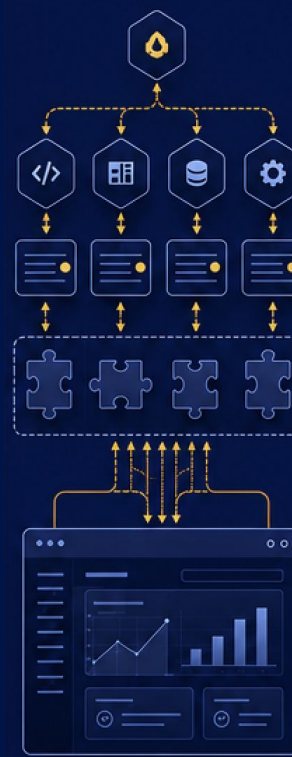
02 The spec

We write the design / product spec live from those.



03 The build

A plan, then parallel agents. Count is whatever the orchestrator spins.



04 The proof

A running AppStore — if compose comes up in the hour.



● THE PROBLEM

AI writes code fast.
It still ships the wrong product.

WITHOUT A SPEC

- The model guesses APIs and page structure
- The running UI drifts from Figma/Design
- Checkout math is invented in the code
- The same prompt yields a different product
- You spend the hour arguing with the model

WITH A SPEC

- Architecture is already agreed in FigJam
- Figma screens are the visual source of truth
- Checkout rules are written before code
- Agents implement one contract
- Done is a store you can click

● WHAT SPEC-DRIVEN DEVELOPMENT IS

The product is a written contract.
Code implements it. Agents do not invent it.

IT IS

Freeze the decisions that would otherwise be guessed — roles, routes, checkout math, done.

IT IS NOT

A longer prompt. Chat is still improvisation.

IT IS NOT

A PRD (Product Requirements Document) nobody reads. If agents can't execute it, it isn't a spec.

TODAY THAT LOOKS LIKE

01

Intent

One prompt. Every question, one pass.

02

Spec

The contract: brand, routes, rules, who can do what.

03

Plan

Independent slices agents can pick up in parallel.

04

Build

Review starts as soon as the first slice lands.

05

Ship

A running store. A small test set keeps the hour moving.

AppStore. An electronics marketplace, not a toy CRUD.

GUEST

Browse home, listings,
product

Search by name and brand

Track an order by number

BUYER

Cart, wishlist, checkout

Discount codes above a
minimum order

Orders and saved addresses

ADMIN

Products and categories

Ten top-level nav items, max

Process and fulfill orders

A complete loop. Not a catalog of every feature.

01

Shopper loop

Browse, search, check out, track an order.

02

Admin loop

List products, manage categories, fulfill.

03

Real enough stack

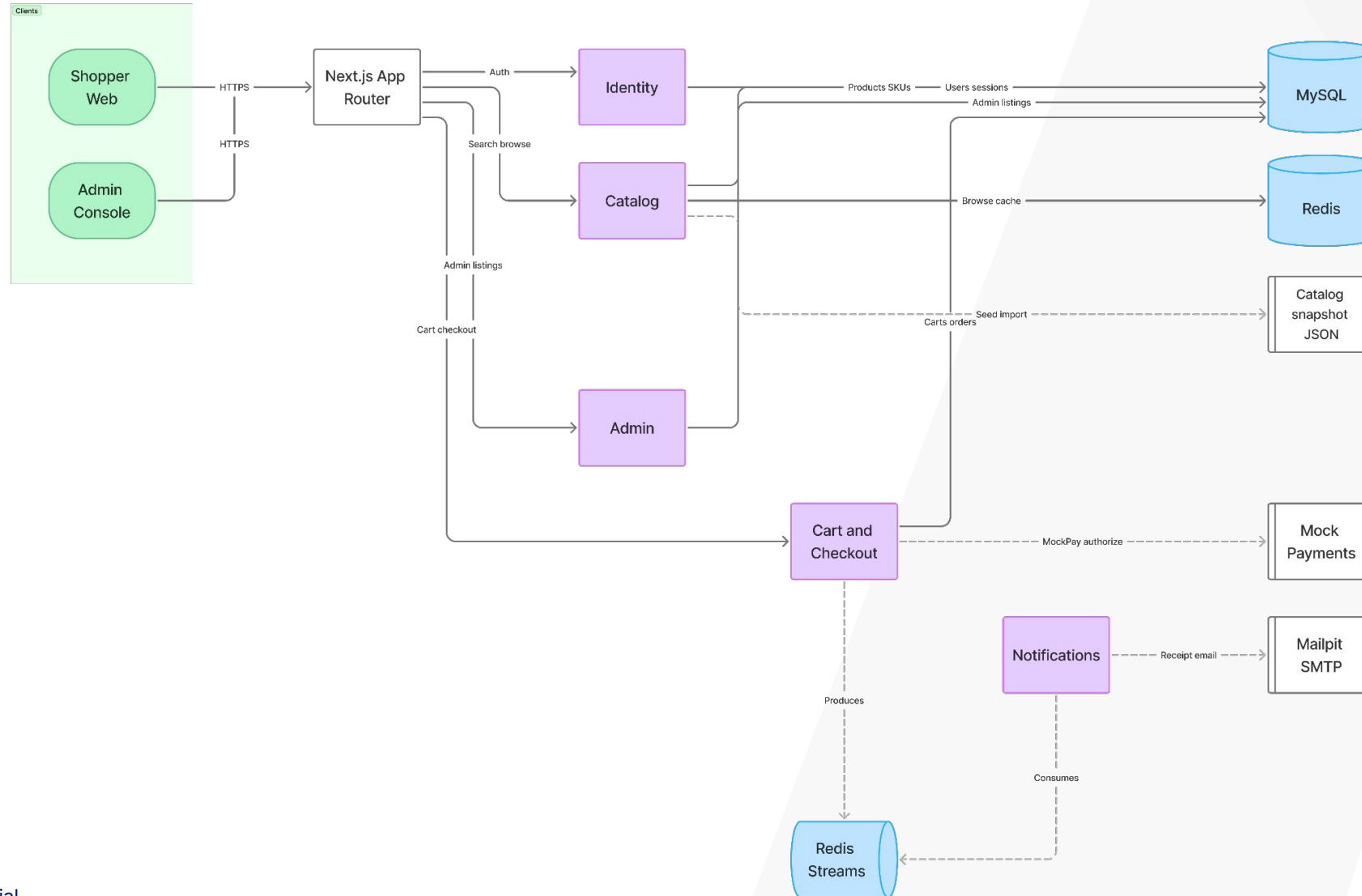
App, database, cache, and email — running locally.

04

Finishable in an hour

Seeded catalog and mock pay so we close the loop on stage.

One app. Four services. This is the board.



What we froze — and what we refused.



Next.js

React only paints the page. Next.js also keeps the database on the server.



MySQL

Stock and the order commit together. Mongo can sell the last unit twice.



Redis

Listings stay fast. The email cannot stall checkout.



Mailpit

We need the receipt on stage, not a bank or a real mailbox.

REFUSED



Not React alone



Not Mongo

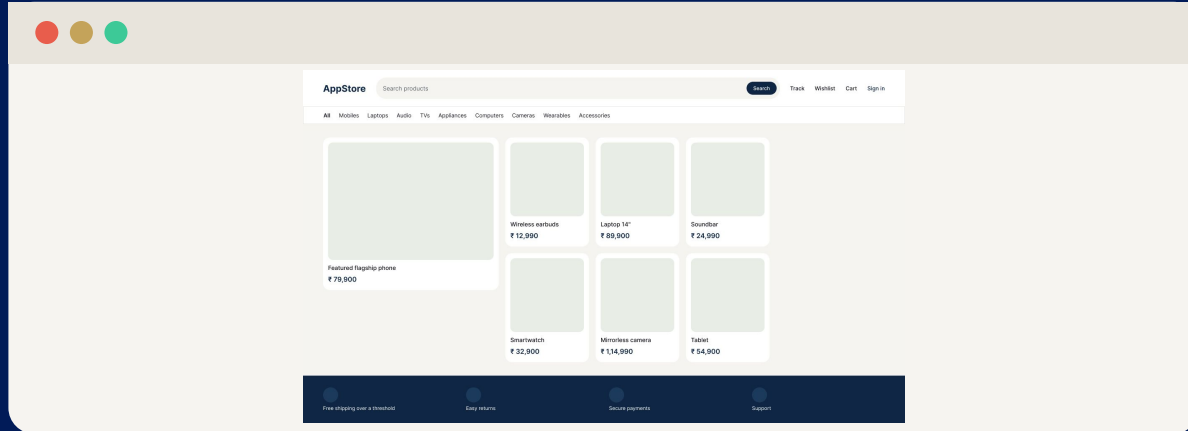


Not a live catalog

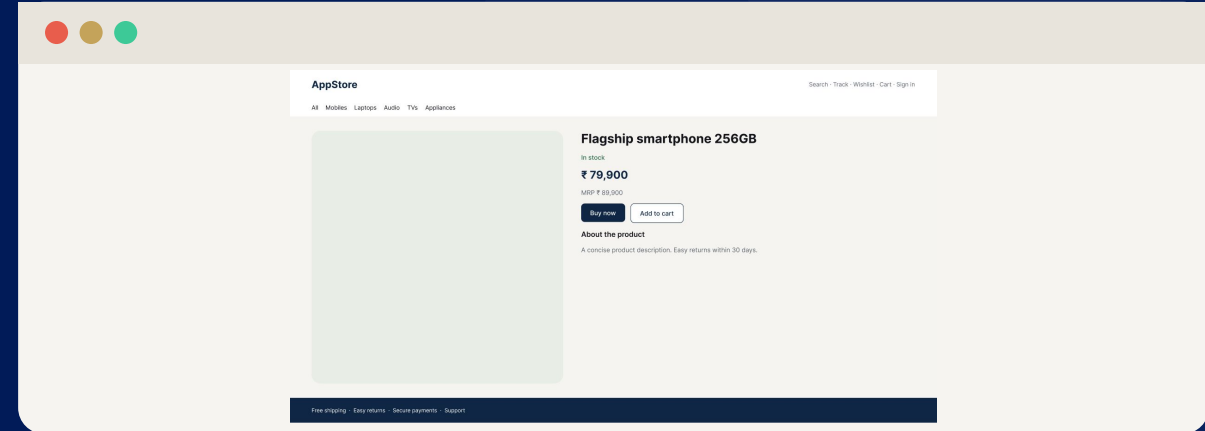


Not a real charge

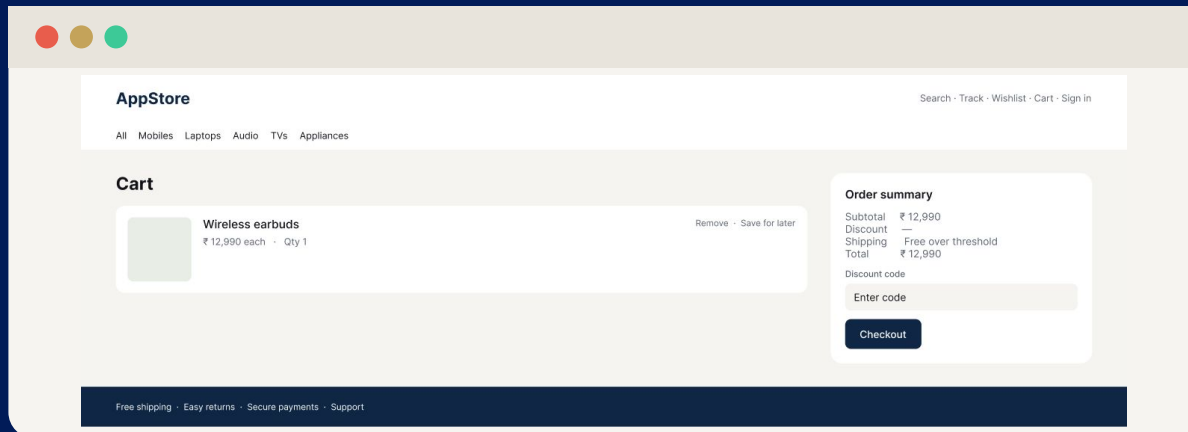
Figma is not inspiration. It is law.



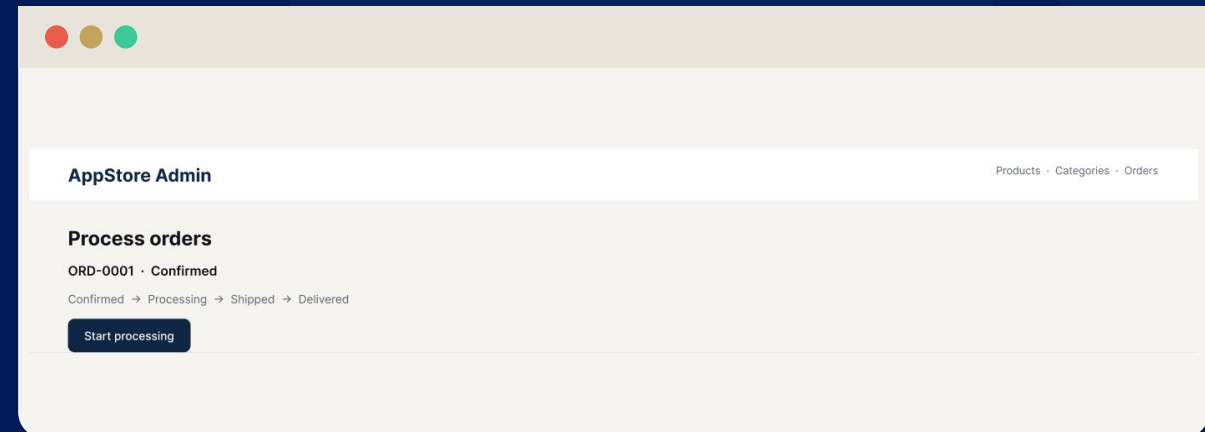
Home mosaic



Product



Cart + summary



Admin · orders

Visual Spec to the Design Spec

Use **Figma MCP** with minimum MCP calls. Pull these once:

- **Figma UI** – <https://www.figma.com/design/npivpYycBDVOsJcfQXd3Pd>
- **FigJam architecture** – <https://www.figma.com/board/ObKdDkvM0Cz0WsiZmxnP36>

Build **AppStore** from those:

- Copy the Figma UI completely.
- Implement the FigJam architecture – **do not invent a new one.**

Start **/brainstorming**. Ask every question together, and only ask further to close a gap.
Do not write app code yet.

After answers:

- write the design spec,
- then continue **/writing-plans** with **4-6 tasks** grouped so independent work can run in parallel.
- **As soon as any task finishes, start the review.**

No TDD novel – only tests for pricing, stock, and discount codes.
Done = docker compose up.

Minimum calls. Maximum signal.



Figma MCP

Pull every screen and the architecture board once. Don't screenshot 19 times by hand.



Skills

Brainstorm → spec → plan → parallel build. The playbook is written down.



Agents

The plan splits independent work. Review starts when the first slice lands.



Compose

Green CI is not the done gate. A browser on :3000 and Mailpit on :8025 is.

Not the task list. The decisions agents must not invent.

Who

Guest, buyer, admin — and which screens each can use.

Catalog

Where products live at runtime, and how they get there.

Checkout

Price, discount, shipping, stock — written as rules, not vibes.

Fulfillment

How an order moves from placed to delivered.

Notify

What happens after an order is confirmed.

Checkout rules — written down so agents don't invent them.



PRICE

Show the selling price. Strike MRP only when it is actually higher.



DISCOUNT

Codes apply to merchandise, not shipping, and only above a minimum order.



SHIPPING

Free above that threshold. A small fee below it.



STOCK

Don't sell more than you have. Reject the extra quantity.



PAY

Shopper picks a method. Authorization is mocked so we can finish the loop.



ACCESS

Guests browse and track. Buyers check out. Admins fulfill.

We won't pre-announce the task list.

01 The plan

Independent slices — whatever the spec actually needs that day.

02 The agents

The orchestrator spins as many as the plan supports. We don't guess the number.

03 The review

Starts when the first slice lands. Everyone else keeps building.

04 The point

Speed without a second architecture sneaking in.

Spec Driven Development in Action

Implement this using `/subagent-driven-development` .

Use **composer-2.5** for implementation and fixing issues, and **Claude Sonnet 5.5 Medium** for reviews.

Group so independent work can run in parallel. Also, as soon as any task finishes, start the review in parallel. Aim is to implement as quickly as possible.



Phase 2: Efficient AI Engineering

AI Tech Meetup India 2026

Yash Sinha & Pratikkumar Mohite | 10th Oct 2026

CORE CLAIM

**AI quality is heavily determined by how we
manage context, not just which model we use.**

Shop catalog review succeeds or fails on **evidence selection** — not on swapping the model.

Shop v1.1 - catalog contract review



Public catalog

Public Visibility

Show only active products to public users

Inactive Handling

Return **404** for inactive public products



Admin vs buyer

Admin Access

Admins can see inactive products

Buyer Restrictions

Buyers blocked on admin endpoints with **403**



catalog.yaml • ProductService.java • SecurityConfig.java • CatalogApiTest.java

01

Why Token Usage Matters

Treat the context window as a budget. Shop catalog review does not need the whole tree.



token budget

Token usage is a product constraint



COST

Every extra file is billed twice

Prompt tokens on the way in.
Completion tokens on the way out.
Dumping Shop's tree multiplies both.

2X BILLING MULTIPLIER



LATENCY

Longer context, slower loops

A catalog review that waits on 160k tokens is a review you will not iterate.

STALLED FEEDBACK LOOPS



CAPACITY

The window is finite

Noise from unrelated Shop modules crowds out ProductService and the tests that prove the rules.

NOISE CROWDS OUT SIGNAL

Same review. Two budgets.

SHOP TREE DUMP

~160k

approx. tokens · 200+ files

Entire Shop output. Most of it is irrelevant to the four catalog rules.

VS

TARGETED EVIDENCE

~5k

approx. tokens · four files

OpenAPI contract, ProductService, SecurityConfig, CatalogApiTest.

Token figures are approximate: character count ÷ 4.

More context is not automatically better context

WHAT DUMPING SHOP DOES

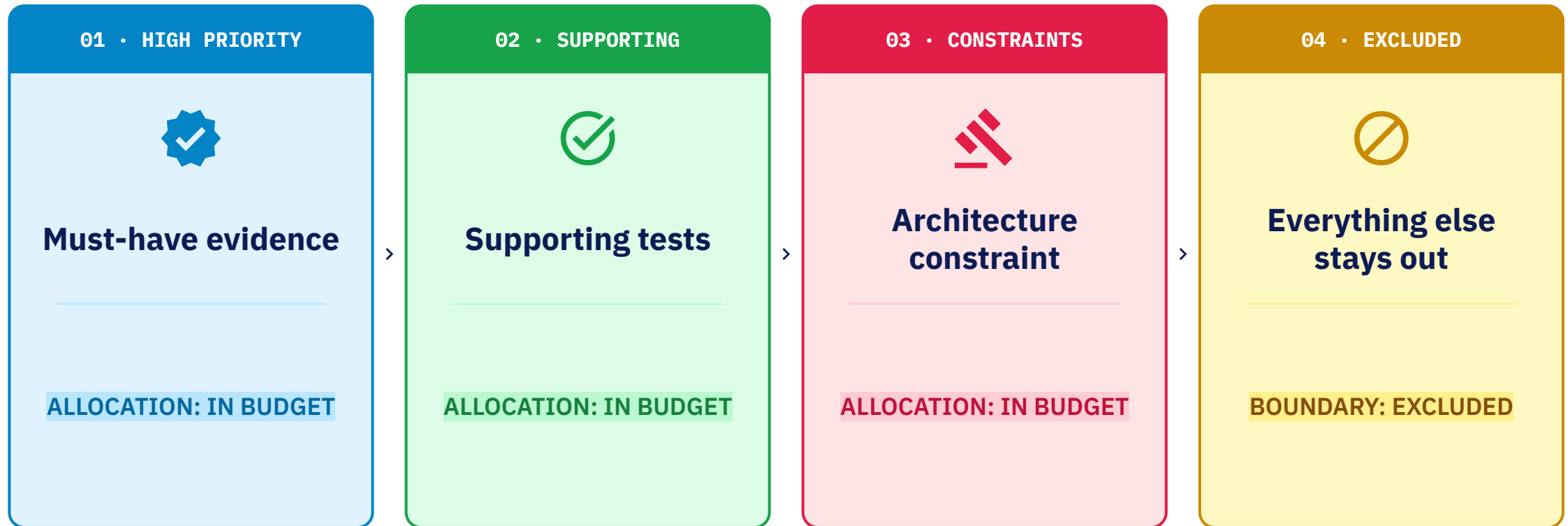
- ✗ Dilutes the four catalog rules
- ✗ Hides searchActive and 404 paths
- ✗ Invites confident, ungrounded verdicts

WHAT SELECTING DOES

- ✓ One rule, one evidence chain
- ✓ Contract → service → security → test
- ✓ Verdicts you can actually check

The context window is a budget, not storage

Treat tokens like working memory. Spend them on the Shop catalog boundary - not on the rest of the tree.



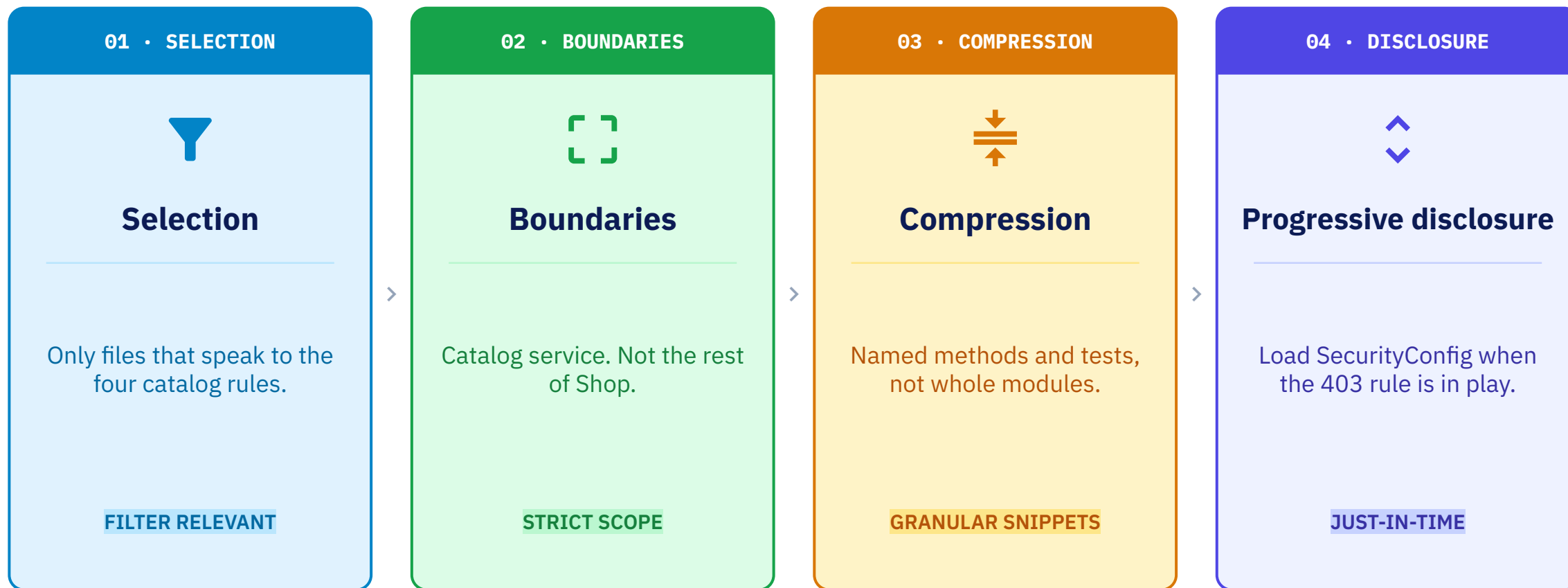
02

Context Engineering

Select, bound, compress, disclose. Spotlight four Shop files — leave the rest dark.

• • ● ● • spotlight

Context engineering



CHECKLIST

What belongs in a Shop catalog review

 **Service Boundary** catalog, not the whole Shop tree

 **Relevant implementation** ProductService.java

 **Test** CatalogApiTest.java, named cases

 **Observed behavior** 404 inactive ProductDetailPage, 403 buyer on admin

 **Architecture constraint** hasRole("ADMIN") in SecurityConfig

Instructions · task context · durable knowledge

01 · INSTRUCTIONS



Instructions

How to review: for every rule, collect contract, impl, security, named test.

WORKFLOW RULES

02 · TASK CONTEXT



Task context

catalog.yaml, ProductService, SecurityConfig, tests.

CURRENT CHANGE

03 · DURABLE KNOWLEDGE



Durable knowledge

Stable conventions: public sees active only; admins see inactive; buyers get 403.

SYSTEM CONVENTIONS

Put each fact in the right layer



Rules

Always-on constraints

Do not invent code paths or test results



Task context

This change only

The four catalog rules



Reference

Looked up on demand

`contracts/openapi/catalog.yaml`



Skills

Reusable review workflow

Catalog contract-review skill



Hooks

Lightweight gates

Require named test before a verdict

03

Skills & Automations

Once a workflow works repeatedly, copy-paste is not enough. Package the procedure.

□ □ □ package

Skill = Procedure + Guardrails

A skill packages the procedure close to the code.

```
SKILL.md — Jira Ticket Refinement

# Skill: Jira Ticket Refinement
- Version: 1.0.0
- Purpose: Automates creation, structuring, & refinement of Agile tickets.

# Workflow Execution Steps
1. Parse input → 2. Extract scope → 3. Generate ACs

# Acceptance Criteria & Implementation
- Technical approach, dependencies, testing strategy

# Constraints & Verification
- Definition of Done validation & risk matrix
```

Open Standard

Frontmatter: Name and relevance triggers.

Body: Workflow steps, constraints, outputs, & verification.

Location

```
.cursor/skills/
├── jira-refinement-skill/
│   ├── SKILL.md
│   ├── references/
│   └── guidelines.md
```

Not a Giant Prompt

Avoid permanent dumps of instructions in static prompts.

Enables Continuous Improvement

Example - vulnerabilities-auto-fix

1

Scope Tickets

Fetch VM Jira tickets for assignees; keep only issues for the current repo

2

Triage & Group

Order by overdue and priority, bucket same root-cause fixes

3

Fix - Parallel

Spawn one worker per fix group; apply code changes on an isolated branch.

4

Verify

Prove patched versions, clear audit/CVE, lint, and tests.

5

Ship and Summarize

Push and open one PR per fix group, manual review list.

vulnerabilities-auto-fix.yml

```
name: vulnerabilities-auto-fix

description: Fix matching VM vulnerability tickets for
current repo.

# Workflow Architecture

# Coordinator: fetch → filter → triage → group
# Workers: fix → audit → test → PR
# Hard stop: verification fails → no push

# Execution Output

# Output: fixed tickets + manual review + PR URLs
```

Trigger a trusted workflow

A Skill makes a workflow reusable. Automation decides when to run it.



Automate the repeatable, preserve judgment

PR quality review

Automated Code Review & Approval Guardrails

1. TRIGGER

Trigger when a PR opens or receives commits.

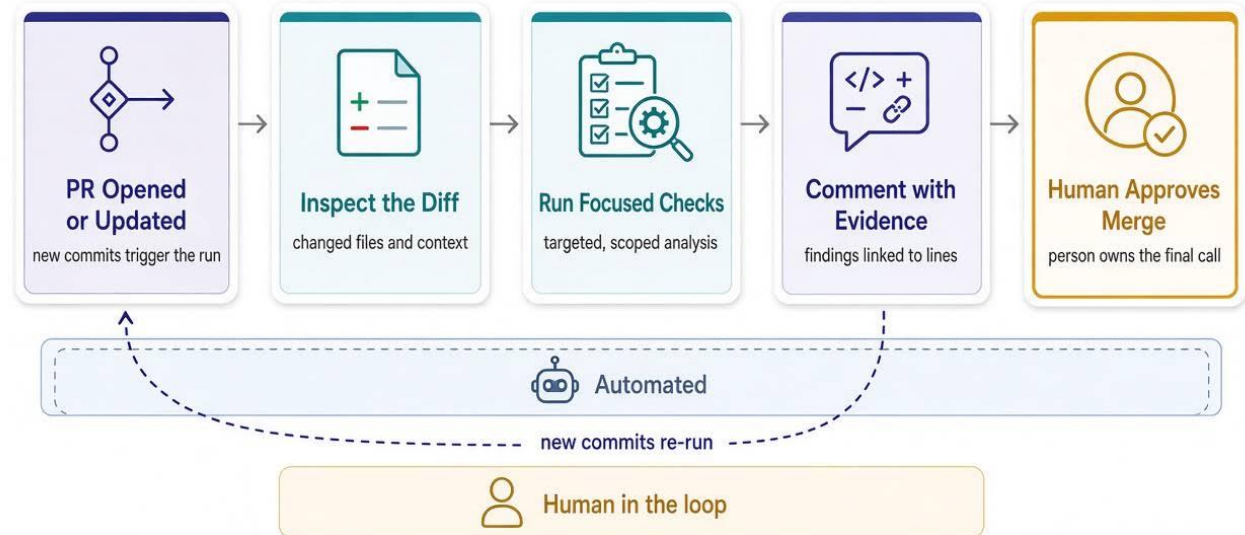
2. INSPECTION & CHECKS

Inspect the diff, run focused checks, and comment with evidence.

3. HUMAN APPROVAL

A human owns final merge approval.

WORKFLOW PROCESS FLOW



Automate the repeatable, preserve judgment

Scheduled maintenance

Automated Repository & Doc Hygiene

1. TRIGGER

Runs on a schedule (nightly or weekly) with no human trigger required.

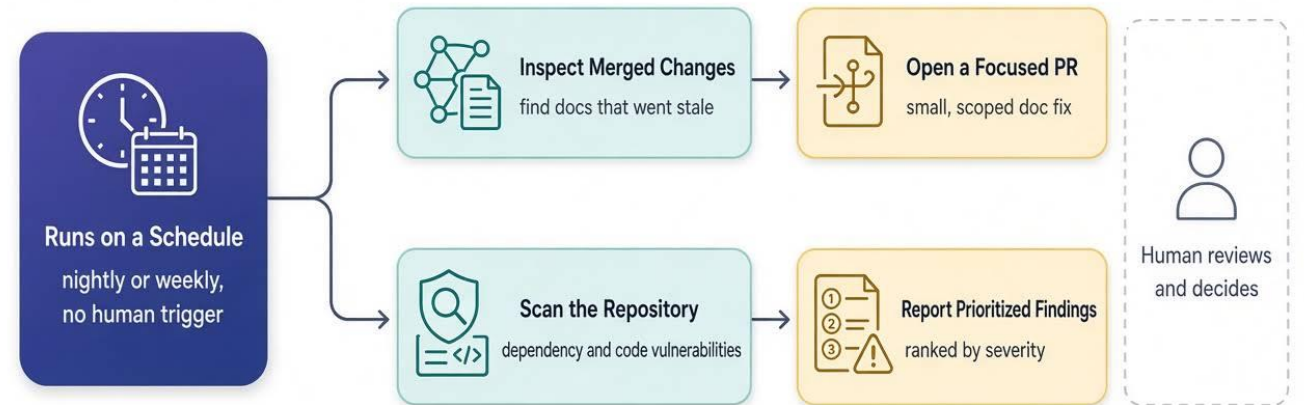
2. INSPECTION & SCANNING

Inspects merged changes for stale docs or scans repository for vulnerabilities.

3. ACTION & DECISION

Opens a focused PR or reports prioritized findings for human review.

WORKFLOW PROCESS FLOW



Automate the repeatable, preserve judgment

Incident Triage

Automated Context Gathering & Escalation

1. TRIGGERS

Slack, PagerDuty, or failed CI pipelines

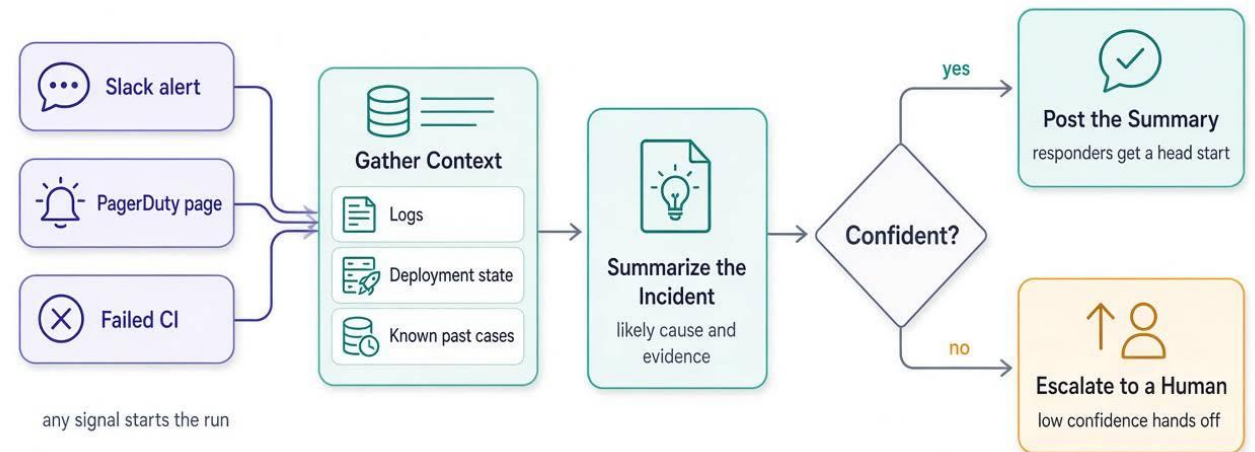
2. DATA COLLECTION

Collects logs, deployment state, and known past cases

3. ESCALATION LOGIC

Summarizes incident & escalates to humans when confidence is low

WORKFLOW PROCESS FLOW



Automate the repeatable, preserve judgment

PR quality review

Trigger when a PR opens or receives commits; inspect the diff, run focused checks, comment with evidence.

A human owns merge approval.

Scheduled maintenance

Inspect merged changes for stale docs or scan a repository for vulnerabilities; open a focused PR or report prioritized findings.

Incident triage

Trigger from Slack, PagerDuty, or failed CI; collect logs, deployment state, and known cases; summarize and escalate when confidence is low.

Agents investigate and prepare, humans approve high-impact actions.

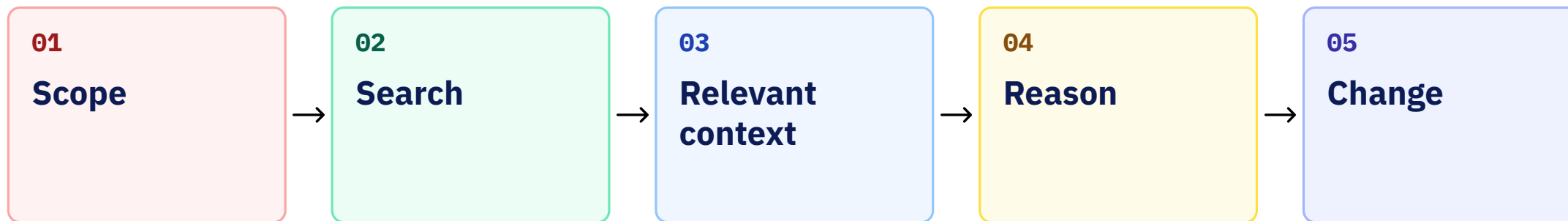
04

Token-Efficient Workflows

Search first. Reason second. Bound what you write — it becomes tomorrow's context.

○—○—○—○ pipeline

Do not start from the whole tree



FOR SHOP

Workflow in Practice

01. Rules First

Start from the four rules to establish immediate context boundaries.

02. Targeted Search

Search ProductService and CatalogApiTest, then reason.

03. Bound Inputs

Never paste 200 files first avoid unnecessary token overhead.

Output is tomorrow's input

PREFER

Prefer



Diffs over full files

Focus changes specifically rather than re-sending full codebases.



Structured verdicts over essays

Provide concise, actionable conclusions instead of lengthy text.



Named methods

`searchActive`, `findByIdAndActiveTrue`

AVOID

Avoid



Re-pasting ProductService wholesale

Avoid dumping large unparsed files into prompt history.



Narrative that hides the 404/403 facts

Do not obscure underlying error statuses behind vague commentary.



Uncited “looks correct” summaries

Always back up claims with exact line references and citations.

Token-efficient Workflow

A practical workflow has four stages. This is not merely a prompting pattern - it is an engineering control loop.

01

Scope

Define the outcome, constraints, and done condition.

02

Retrieve

Open the smallest set of likely-relevant sources.

03

Reason

Form a hypothesis and make one coherent change. Do not rediscover the same facts.

04

Verify

Run focused tests, inspect the diff, and expand only when evidence requires it.

Token-efficient Workflow

A practical workflow has four stages. This is not merely a prompting pattern - it is an engineering control loop.

01

Scope

Define the outcome, constraints, and done condition.

Was the outcome ambiguous?

02

Retrieve

Open the smallest set of likely-relevant sources.

Was evidence missing?

03

Reason

Form a hypothesis and make one coherent change. Do not rediscover the same facts.

Did reasoning drift?

04

Verify

Run focused tests, inspect the diff, and expand only when evidence requires it.

Did checks fail to catch it?

Token-efficient Workflow

A practical workflow has four stages. This is not merely a prompting pattern - it is an engineering control loop.

01

Scope

Define the outcome, constraints, and done condition.

Was the outcome ambiguous?

02

Retrieve

Open the smallest set of likely-relevant sources.

Was evidence missing?

03

Reason

Form a hypothesis and make one coherent change. Do not rediscover the same facts.

Did reasoning drift?

04

Verify

Run focused tests, inspect the diff, and expand only when evidence requires it.

Did checks fail to catch it?

Question: *How do we know which stage actually created the waste?*

05

Observability : From intuition to evidence

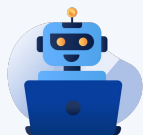
An AI agent is a running software system, and running software systems can be observed.

◉ ◉ ◉ observability

We cannot improve what we cannot see.

Metrics

Quantitative measurements of an agent turn



How fast?

How many?

How much?

Agent Turn Metrics

Token count: How many input / output tokens?

Tool volume: How many Read / Grep / shell calls?

Duration: How long did the turn take?

Loop count: How many agent loops before stop?

Failure rate: What share of tool calls failed?

Types of Metrics



Counters

Monotonically increasing. They only go up.

Examples: total tokens consumed, total tool calls, total errors.



Gauges

Point-in-time values that go up or down.

Examples: in-flight turns, queue depth, active agents.



Histograms

Distributions, not just averages.

Examples: p95 turn duration, token percentiles by model.

Logs

Discrete records of events

Exact Tools

Record specific tool execution:
Read, Grep, Shell, Apply patch.

Execution Context

Capture command, exit code,
stderr, touched files, and retry
counts per turn.

Dual Formatting

Terminal streams for humans;
structured JSON (repo, model,
trace_id) for analytics.

Actionable Levels

INFO for tool calls, **WARN** for
retries, **ERROR** for failures.

Human-readable logs

```
terminal.log

14:32:10 INFO grep "auth" src/** hits=42
14:32:11 WARN tool Read retried 3/3
14:32:11 ERROR shell npm test exit 1
```

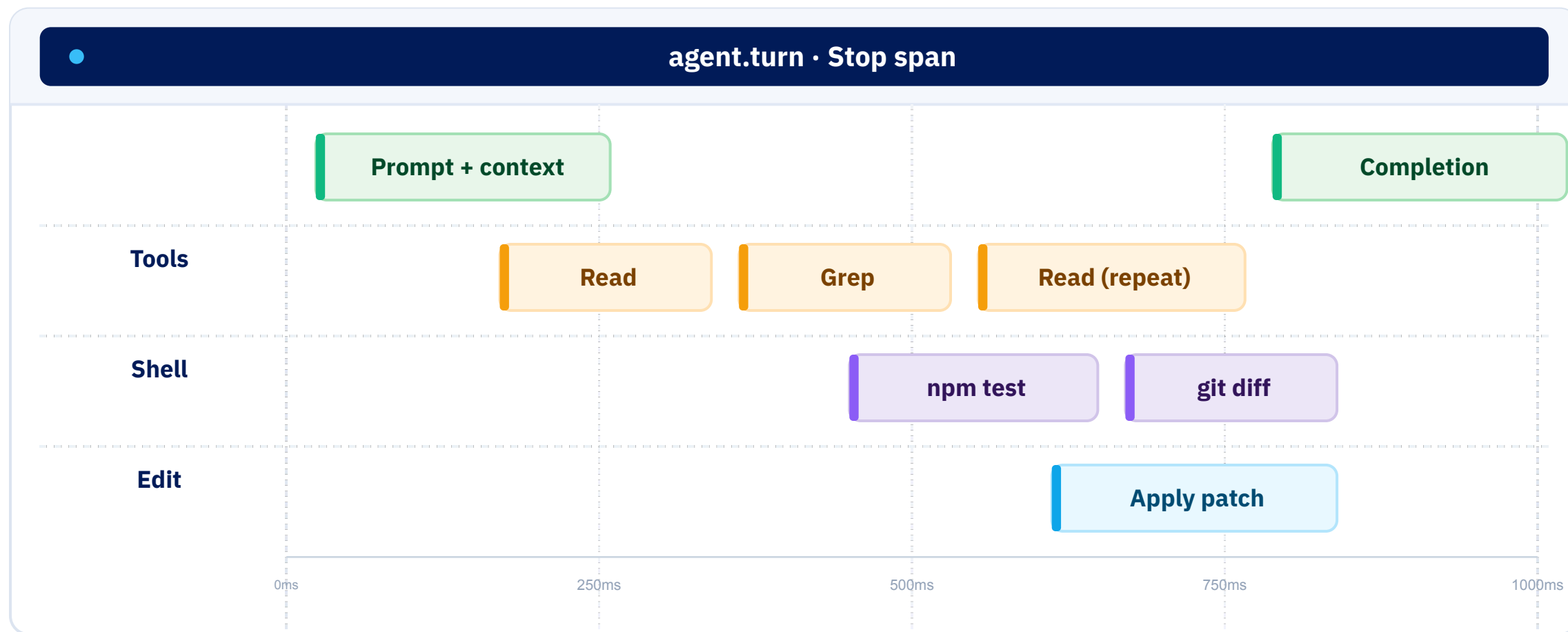
Machine-readable (structured) logs

```
event.json

{
  "timestamp": "2024-01-15T14:32:10Z",
  "level": "INFO",
  "message": "tool grep completed",
  "hits": 42,
  "repo": "payments-api",
  "trace_id": "abc123"
}
```

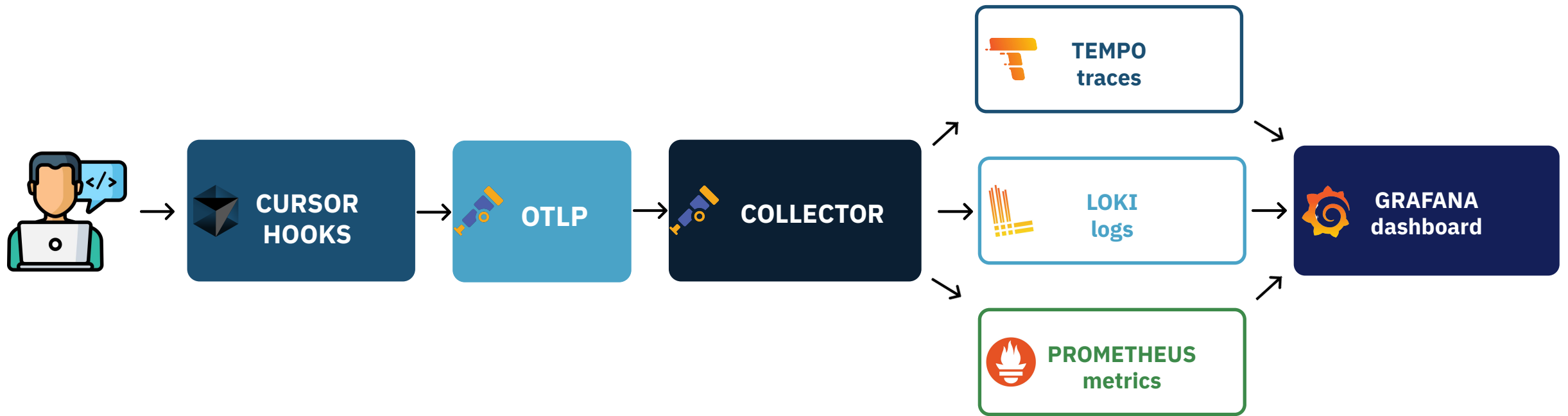
Traces

The complete journey of an agent turn through the system



Cursor OTEL Pipeline

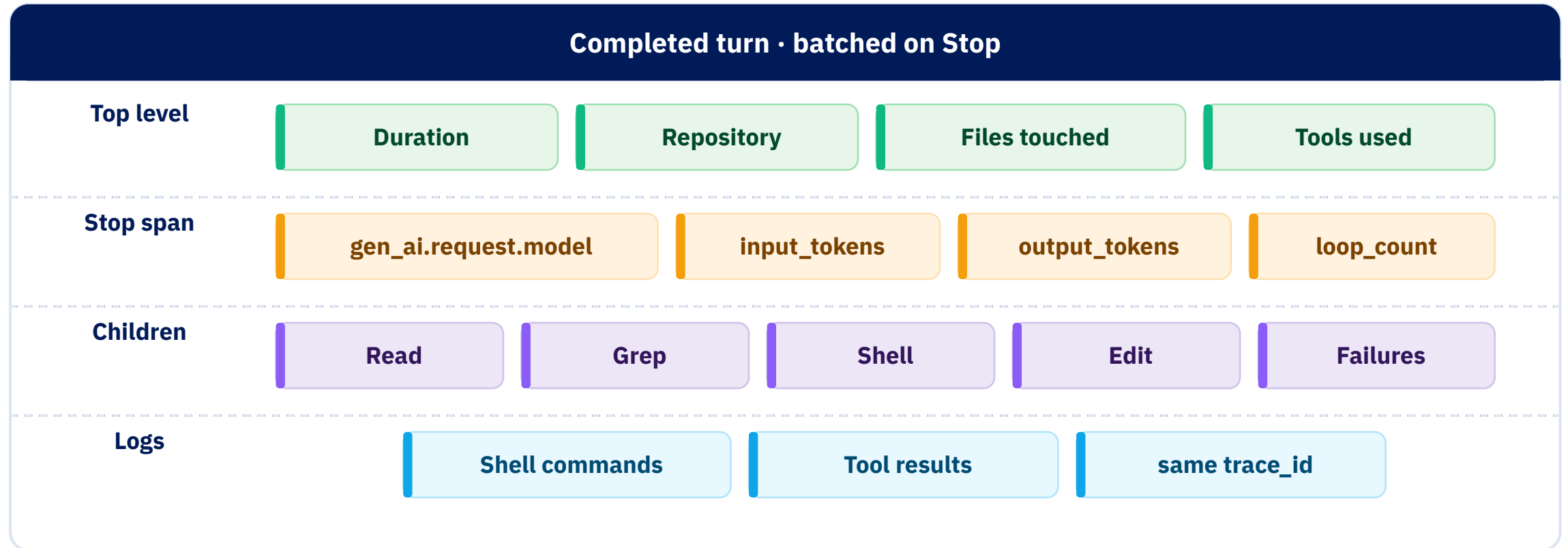
Signals leave the laptop, fan out in the collector, and come back to the same developer in Grafana.



Correlate a turn, inspect its tools, aggregate its cost without relying only on the chat window.

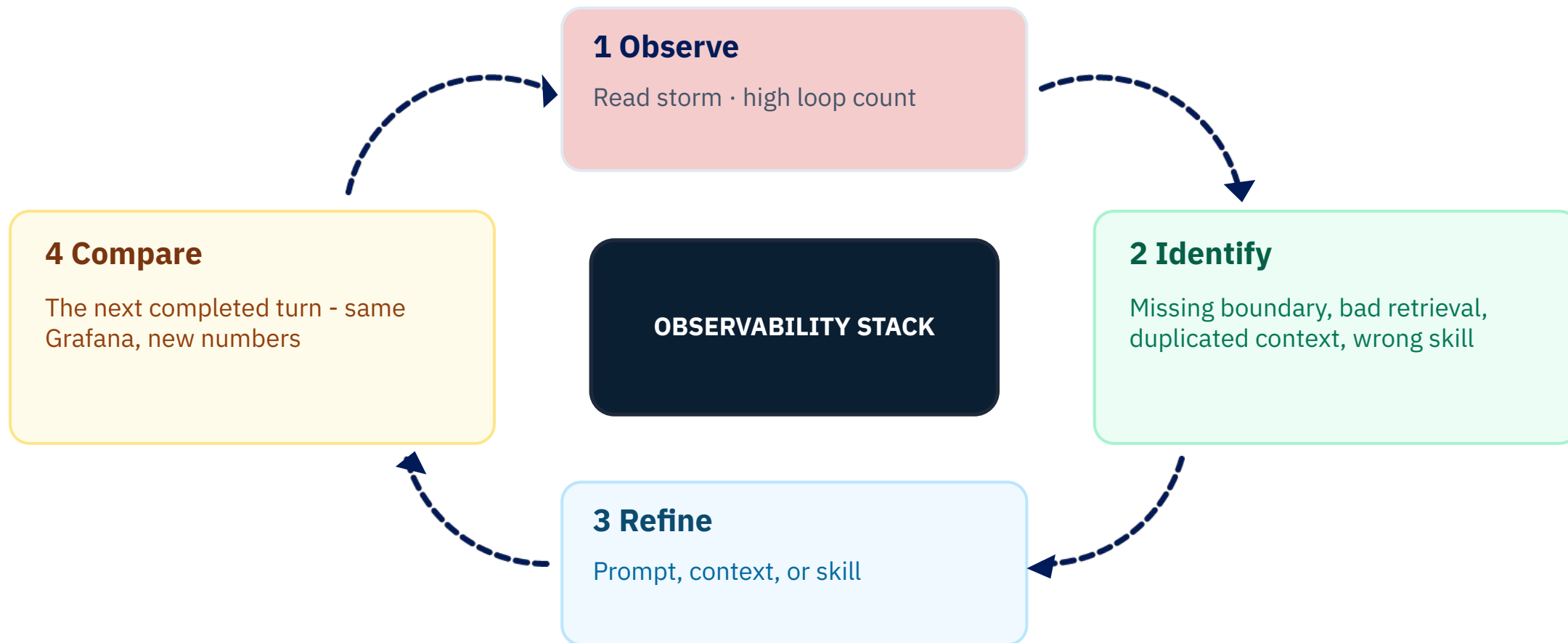
What one agent turn reveals

One completed turn is a compact engineering record - layered like a trace, joined by one Trace ID.



Telemetry is batched when the agent stops - inspect completed turns, not a half-finished one.

Close the efficiency loop



Observability is the hub, each pass should change the next turn.

ORG-Level Alerting

Admins can wire Grafana or the collector to policy - not a developer dashboard.

01

Per User Token Cap

Alert when a person or service account crosses a monthly token budget.

02

Unusual Model Mix

Spike in Opus/Expensive models vs. the team's normal Composer baseline.

03

Sensitive Input

Secrets, PII, or customer payloads detected in prompts or tool args.

04

Cost or Duration Burst

One repo or hour far above p95 spend or turn length.

05

Policy/Tool Abuse

Repeated shell failures, blocked paths, or tools outside the approved set.

Five takeaways

01 **More context is not automatically better.** Relevant context wins.

02 **A precise prompt can reduce total work,** even when it contains more words.

03 **Observe the workflow, not only the answer.** Tokens, tools, traces, and logs reveal waste.

04 **Package successful workflows as skills.**

05 **Automate only repeatable workflows** with explicit boundaries.

Four beats. No filler.

01 The inputs

Architecture board and UI screens — already designed.



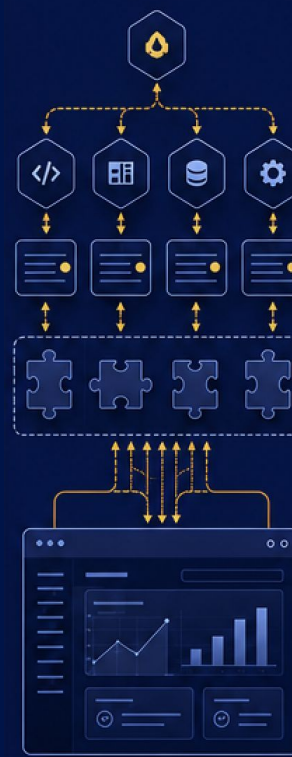
02 The spec

We write the design / product spec live from those.



03 The build

A plan, then parallel agents. Count is whatever the orchestrator spins.



04 The proof

A running AppStore — if compose comes up in the hour.



Prompt we used

Implement this using **/subagent-driven-development** .

Use **composer-2.5** for **implementation and fixing issues**, and **Claude Sonnet 5.5 Medium** for **reviews**.

Group so independent work can run in parallel. Also, as soon as any task finishes, start the review in parallel. Aim is to implement as quickly as possible.

Relay race, or divide and conquer?

01 The Relay Race



One person finishes their piece, then hands off to the next. Everyone else waits for the baton.

Sequential hand-offs & dependent workflow

02 Divide & Conquer



Work that doesn't depend on anything starts immediately.

Example: A UI engineer builds against mock data while the backend is still in progress.

One Prompt. Many subagents. Zero idle time.



Composer 2.5 implements and fixes. **Claude Sonnet 5.5 medium** reviews. Each model plays to its strength.



Independent work is grouped and kicked off together — no one waits on a task they don't depend on.



The moment a task lands, its review starts — not after the whole batch finishes.



Implement → review → fix, on repeat, until everything passes. No human input unless truly blocked.

Why Subagent Driven Development works.



Isolated context — each subagent gets only the task at hand, never the coordinator's full history, so it stays focused and never gets confused.



Continuous review — the moment a task lands, its review starts. No pile-up waiting for the whole batch to finish.



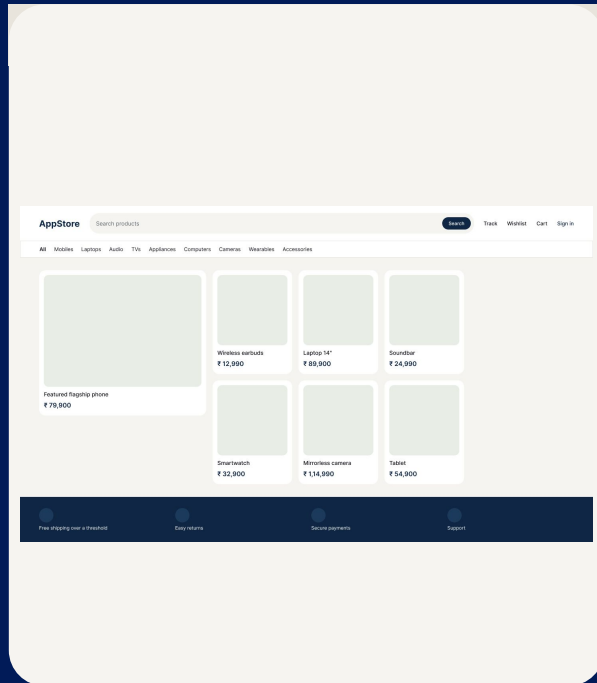
Right model, right role — implementation and review are each matched to the model best suited for that job.



No idle time — subagents keep working in parallel until everything is built, reviewed, and passed, without waiting on a human.

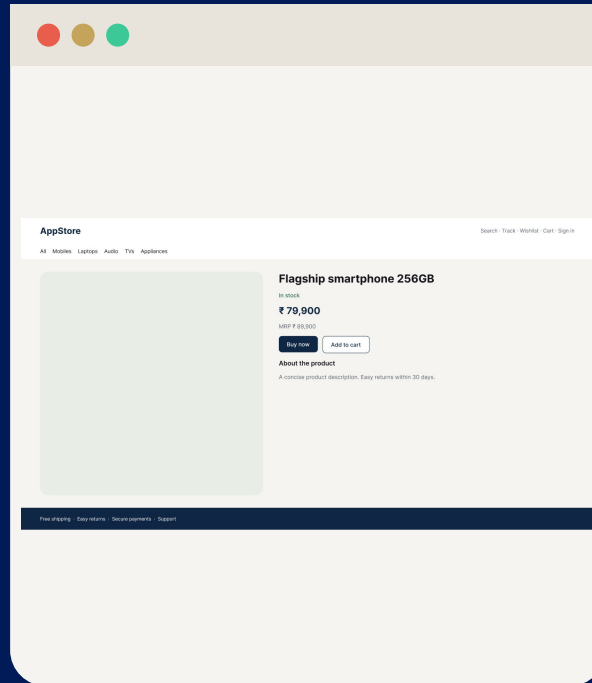
● IF THE STORE COMES UP

We'll walk the spec in the browser — not a feature tour.



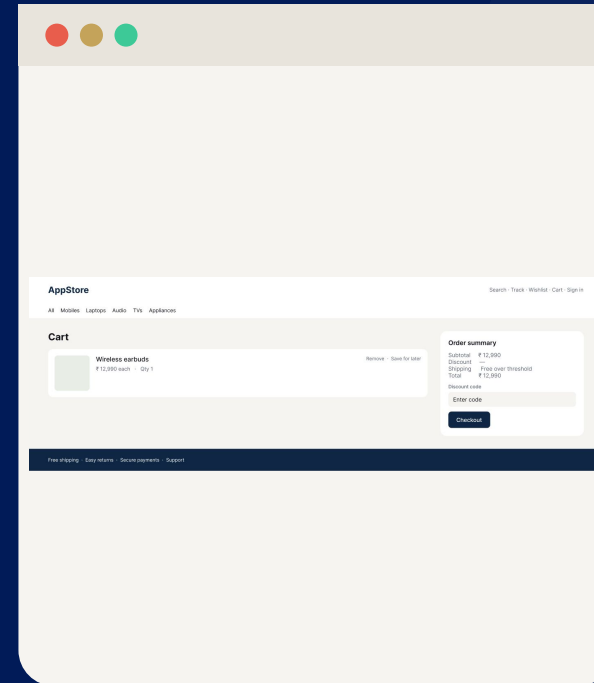
Home

Chrome and catalog match Figma.



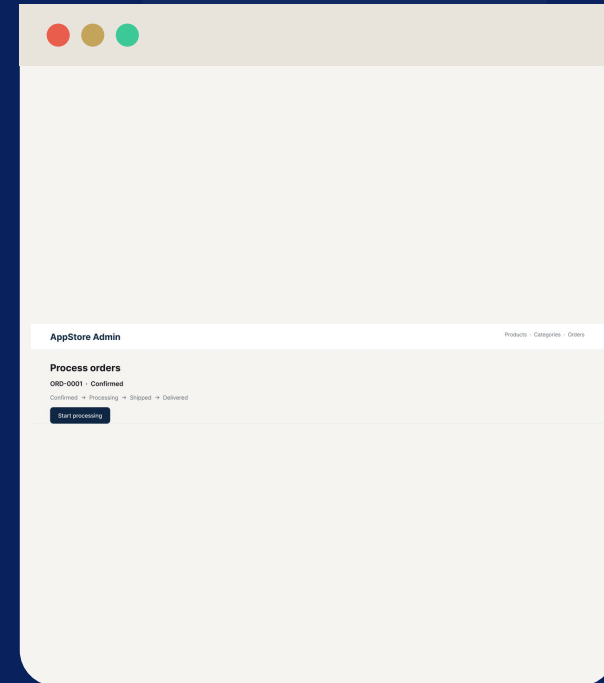
Product

Price, stock, and actions as specified.



Cart

A discount and a placed order.



Admin

The order can be fulfilled.

Five laws to steal on Monday.



Spec before code

If you didn't approve it, agents will invent it.



Ask once

One question pass. Then lock. Chat is not a spec.



Name the seams

Cookies, streams, enums, helpers — freeze them.



Review as a side channel

Start when the first task finishes, not at the end.

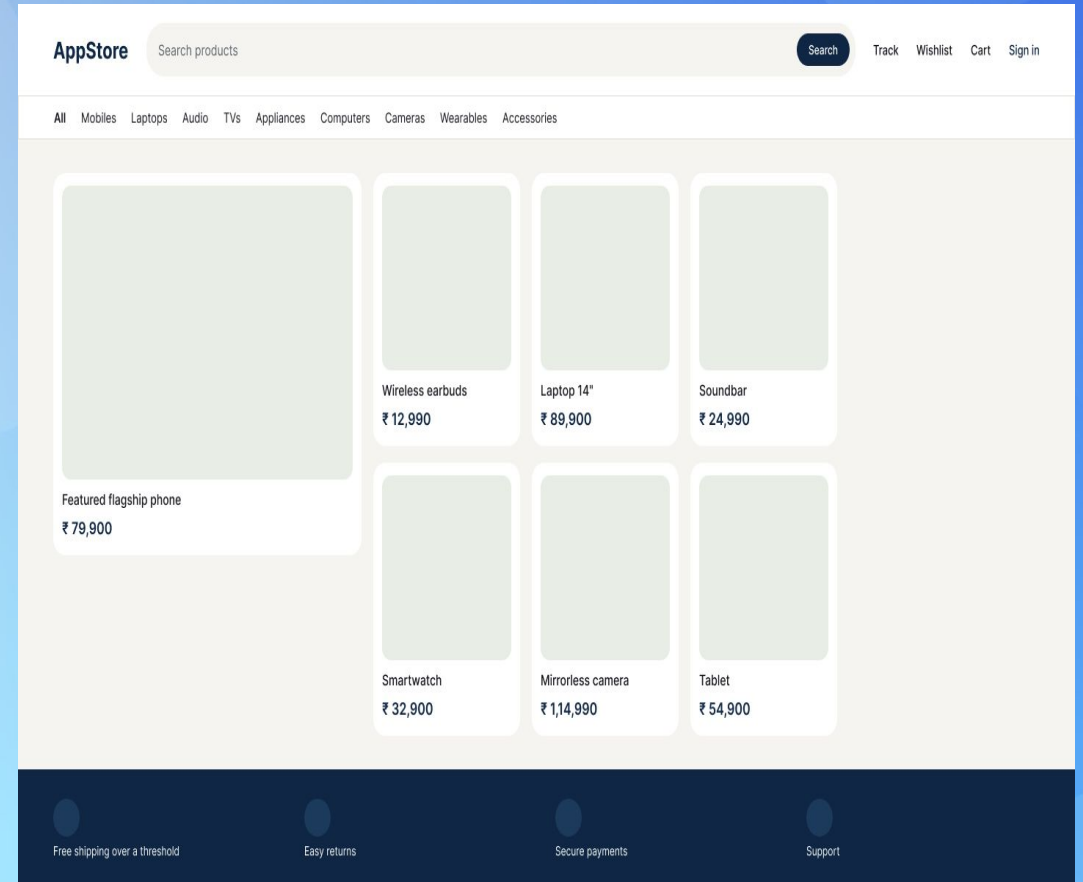


Done is running

A small test set keeps the live hour moving. Full coverage when you have time.

THE ASK

**Pick one feature this week.
Write the spec before you open the
agent.**



Thank You

